Introduction to Computer Science
Fall 2026
Homework 3

Reference Solution

Problem 1

(a) The three binary fractional digits give the values

$$(0.000)_2 = 0, \quad (0.001)_2 = \frac{1}{8}, \quad (0.010)_2 = \frac{1}{4}, \quad (0.011)_2 = \frac{3}{8},$$
$$(0.100)_2 = \frac{1}{2}, \quad (0.101)_2 = \frac{5}{8}, \quad (0.110)_2 = \frac{3}{4}, \quad (0.111)_2 = \frac{7}{8}.$$

Thus, the representable numbers between 0 and 1, inclusive, are

$$0,\ \frac{1}{8},\ \frac{1}{4},\ \frac{3}{8},\ \frac{1}{2},\ \frac{5}{8},\ \frac{3}{4},\ \frac{7}{8}.$$

(b) Every representable number is an integer multiple of $1/8$, but $0.3 \times 8 = 2.4$ is not an integer. Therefore, 0.3 cannot be represented exactly. The two nearest candidates are

$$\left|0.3 - \frac{1}{4}\right| = 0.05, \qquad \left|0.3 - \frac{3}{8}\right| = 0.075.$$

Hence, the closest representable number is

$$\boxed{\frac{1}{4} = (0.010)_2 = 0.25}.$$

(c) Yes, addition is associative in this number system. Every stored number is an integer multiple of $1/8$, so adding two stored numbers produces another multiple of $1/8$. Storing the result discards the integer part, which is equivalent to taking the fractional part. Define $\mathrm{frac}(x) = x - \lfloor x \rfloor$. The stored addition operation is therefore

$$a \oplus b = \mathrm{frac}(a + b).$$

Consequently,

$$\begin{aligned}(a \oplus b) \oplus c &= \mathrm{frac}\big(\mathrm{frac}(a+b) + c\big) \\ &= \mathrm{frac}(a + b + c) \\ &= \mathrm{frac}\big(a + \mathrm{frac}(b+c)\big) \\ &= a \oplus (b \oplus c).\end{aligned}$$

Thus, $(a + b) + c = a + (b + c)$ always holds under the specified storage rule.

(d) No. Multiplication is not always associative because fractional bits beyond the third are truncated after each operation. For example, let

$$a = \frac{1}{4}, \qquad b = \frac{5}{8}, \qquad c = \frac{7}{8}.$$

First, $ab = 5/32 = (0.00101)_2$, which is stored as $(0.001)_2 = 1/8$. Then

$$(ab)c \longrightarrow \frac{1}{8} \cdot \frac{7}{8} = \frac{7}{64} = (0.000111)_2 \longrightarrow (0.000)_2 = 0.$$

On the other hand, $bc = 35/64 = (0.100011)_2$, which is stored as $(0.100)_2 = 1/2$. Therefore,

$$a(bc) \longrightarrow \frac{1}{4} \cdot \frac{1}{2} = \frac{1}{8} = (0.001)_2.$$

Hence,

$$\boxed{(ab)c = 0 \neq \frac{1}{8} = a(bc)}.$$

Problem 2

(a)

Instruction	Direct data dependency/dependencies
I_2	I_1
I_3	I_2
I_4	None
I_5	None
I_6	I_4, I_5
I_7	I_3, I_6

(b) With in-order execution, the earliest unexecuted instruction blocks all later instructions if its inputs are not available. The schedule is therefore:

Cycle	Instruction 1	Instruction 2
1	I_1	
2	I_2	
3	I_3	I_4
4	I_5	
5	I_6	
6	I_7	

All seven instructions complete in $\boxed{\text{6 cycles}}$.

(c) With out-of-order execution, independent instructions can be scheduled while respecting data dependencies:

Cycle	Instruction 1	Instruction 2
1	I_1	I_4
2	I_2	I_5
3	I_3	I_6
4	I_7	

A processor that executes one instruction per cycle takes seven cycles. Thus, the speedup is

$$\boxed{\frac{7}{4} = 1.75}.$$

(d) The minimum is $\boxed{4 \text{ cycles}}$. For example, a valid schedule for a processor with width four is

Cycle	Instructions executed
1	I_1, I_4, I_5
2	I_2, I_6
3	I_3
4	I_7

This is optimal because the dependency chain $I_1 \to I_2 \to I_3 \to I_7$ contains four instructions. Since each instruction's result is available only at the beginning of the next cycle, these four instructions must occupy at least four consecutive cycles, regardless of how many instructions can execute in parallel.

Problem 3

3.1 A direct-mapped cache

(a) The cache state after each request is:

Step	Requested block	Hit/Miss	Line 0	Line 1	Line 2	Line 3
1	0	M	0	–	–	–
2	1	M	0	1	–	–
3	2	M	0	1	2	–
4	3	M	0	1	2	3
5	0	H	0	1	2	3
6	4	M	4	1	2	3
7	0	M	0	1	2	3
8	1	H	0	1	2	3
9	5	M	0	5	2	3
10	1	M	0	1	2	3

There are two hits in ten requests, so the hit rate is

$$\boxed{\frac{2}{10} = 20\%}.$$

3.2 Why store a block instead of one value?

(b) The hit/miss sequences are:

Cache A: $M,\ M,\ M,\ M,\ M,\ M,\ M,\ M,$
Cache B: $M,\ H,\ H,\ H,\ M,\ H,\ H,\ H.$

In Cache A, each requested word is a separate block, and all eight words are different, so every request misses. Its hit rate is $\boxed{0/8 = 0\%}$. In Cache B, requesting word 0 loads the block containing words $0, 1, 2, 3$, and requesting word 8 loads the block containing words $8, 9, 10, 11$. The other six requests hit. Its hit rate is $\boxed{6/8 = 75\%}$. Cache B exploits spatial locality by fetching nearby words together, even though both caches hold 16 words in total.

3.3 When blocks fight for the same place

(c) For the request sequence $0, 4, 0, 4, 0, 4$, blocks 0 and 4 both map to line 0 in the direct-mapped cache. They continually evict one another, giving the sequence

$$\text{Direct-mapped: } M,\ M,\ M,\ M,\ M,\ M$$

and a hit rate of $\boxed{0\%}$.

In the 2-way set-associative cache, both blocks map to set 0, but that set can hold two blocks. Thus, after the first two misses, both blocks remain in the cache:

$$\text{2-way set-associative: } M,\ M,\ H,\ H,\ H,\ H.$$

Its hit rate is $\boxed{4/6 = 2/3 \approx 66.7\%}$. Although the total capacity is the same, set associativity allows blocks that compete for the same direct-mapped line to coexist in a set.

3.4 Is a lower miss rate always better?

(d) When $P = 50$ ns, the average access times are

$$T_X = 1 + 0.10(50) = \boxed{6 \text{ ns}},$$
$$T_Y = 2 + 0.05(50) = \boxed{4.5 \text{ ns}}.$$

Cache Y is faster when

$$2 + 0.05P < 1 + 0.10P \iff 1 < 0.05P \iff \boxed{P > 20 \text{ ns}}.$$

At $P = 20$ ns, the caches have equal average access time. When $0 < P < 20$ ns, Cache X is faster. A lower miss rate does not necessarily imply a faster cache because a cache with a lower miss rate may have a longer hit time; whether it wins depends on the miss penalty.